Online Bookstore — Security & Integration Plan (M4)

Owner: M4 (Security / Integration) · **Inputs:** SRS v1.0 Draft, SAD v1.0 Draft · **Status:** Draft for team review

This document covers my deliverables: security objectives, security requirements, security architecture, the use-case diagram, and the final consistency check across the SRS, SAD and test plan. Items marked **[Proposed]** need team confirmation.

---

1. Security Objectives

The SRS leaves these as TBD, so I propose them here for M1 to merge into the SRS.

ID	Objective	Meaning for this system
SO-1	**Confidentiality**	A user can only see their own cart, orders and account data. Passwords are never recoverable.
SO-2	**Integrity**	Prices, totals, roles and ownership are decided by the server, never trusted from the client. Orders are all-or-nothing.
SO-3	**Availability**	Misuse (e.g. expensive search spam) should not take the site down. Supports the SRS 99% target.
SO-4	**Authenticity**	Every protected action is tied to a verified, logged-in identity.
SO-5	**Authorization**	Customer and Seller can do only what their role permits, enforced on the server.
SO-6	**Accountability**	Failed logins, denied access and failed orders are logged (without secrets) so incidents can be investigated.

---

2. Security Requirements

Each requirement is testable, has a stable ID, and maps back to an SRS area (SAD §3.8). M3 can reference these IDs directly in test cases.

ID	Requirement	Objective	Verification
SEC-01	Passwords shall be stored only as salted hashes using Django's password hasher.	SO-1	Inspect DB; no plaintext.
SEC-02	Password validators shall be enabled (min length, not common, not all-numeric).	SO-4	Register with weak password → rejected.
SEC-03	Login failures shall return one generic message that does not reveal whether the username exists.	SO-1	Compare wrong-user vs wrong-password responses.

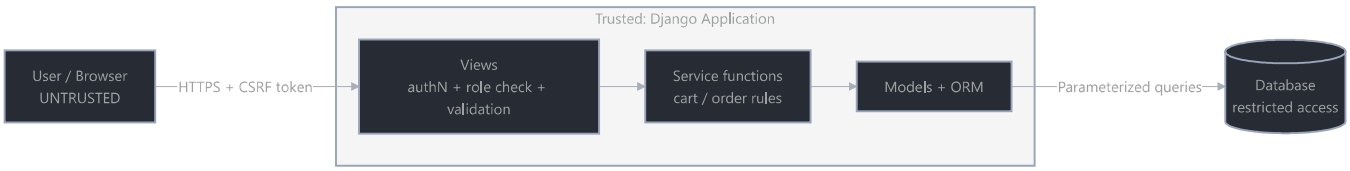
SEC-04	Sessions shall be created only after valid credentials, regenerated at login, and destroyed at logout.	SO-4	Session key changes on login; old session is invalid after logout.
SEC-05	Every protected route shall require authentication. Unauthenticated users are redirected to login.	SO-4	Anonymous request to each protected route.
SEC-06	Every protected route shall enforce role on the server (RBAC matrix, §3.3). Wrong role → HTTP 403 with no data change.	SO-5	Customer hits `/books/add/`; Seller hits `/cart/add/`.
SEC-07	The role shall be stored server-side and chosen from an allowed list (Customer/Seller). It shall never be taken from a hidden field or URL, and users cannot become staff/admin via any form. **[Proposed]**	SO-5	Tamper with POST data.
SEC-08	Cart and order queries shall be scoped to `request.user`. Client-supplied customer IDs are ignored.	SO-1	User A requests user B's cart item or order → 404.
SEC-09	Prices and totals shall be computed from stored book records. Client-submitted prices are ignored.	SO-2	POST tampered price → total unchanged.
SEC-10	Quantities shall be validated as positive integers within a defined maximum.	SO-2	Quantity 0, -1, 99999, "abc".
SEC-11	Order creation (Order + OrderItems + cart clearing) shall run in one database transaction.	SO-2	Force failure mid-way → no partial order.
SEC-12	All state-changing requests shall use POST with CSRF tokens.	SO-2	POST without token → 403.
SEC-13	User-supplied content (book title/description, search text) shall be output-escaped. Raw-HTML rendering is not allowed.	SO-2	Submit `<script>` in a listing → shown as text.
SEC-14	Database access shall use the ORM only, with no string-built SQL.	SO-2	Code review / grep for `raw(` and `execute(`.
SEC-15	Search input shall have a maximum length and result pagination.	SO-3	Very long query and large result set.
SEC-16	Repeated failed logins shall be throttled (e.g. 5 attempts then temporary lockout). **[Proposed, optional]**	SO-3/4	Script 6 bad logins.
SEC-17	In hosted environments: `DEBUG=False`, HTTPS, secure and HttpOnly cookies, HSTS. Secrets come	SO-1	Django `check --deploy`.

	from environment variables, not the repo.		
SEC-18	Logs shall never contain passwords, session IDs, secret keys or tokens. Failed login, denied access and failed order events are logged.	SO-6	Log review.
SEC-19	Users shall see generic error pages. Stack traces are never exposed.	SO-1	Trigger a 500 and a 404.
SEC-20	The post-login `next` redirect parameter shall be validated against the host (no open redirect).	SO-4	`?next=https://evil.com` → ignored.

---

3. Security Architecture

3.1 Trust boundaries



Principle: everything crossing the browser→server line is untrusted input. Validation and authorization happen at the view layer, and the business rules (prices, totals, ownership) are recomputed in the service layer.

3.2 Defence in depth

Layer	Controls
Presentation	Template auto-escaping, CSRF token in every form, hide irrelevant buttons (UX only, not security).
Request/Access	`login_required`, a custom `role_required("SELLER")` decorator, form validation, safe redirects.
Feature	Ownership filters (`filter(customer=request.user)`), server-side price lookup, `transaction.atomic()`.
Data	ORM parameterization, DB constraints (FK, NOT NULL, `CHECK(quantity > 0)`, unique `(customer, book)` on CartItem).
Deployment	HTTPS, env-based secrets, least-privilege DB user, backups, `DEBUG=False`.

3.3 RBAC matrix (single source of truth)

Operation (route)	Anonymous	Customer	Seller
Register / Login	✅	✅	✅

Browse / search (`/books/` , `/books/search/`)	✅	✅	✅
Add listing (`/books/add/`)	❌ → login	❌ 403	✅
View cart (`/cart/`)	❌ → login	✅ own	❌ 403
Add / remove cart item	❌ → login	✅ own	❌ 403
Place order (`/orders/place/`)	❌ → login	✅ own	❌ 403
View order history (`/orders/`)	❌ → login	✅ own	❌ 403

Every row maps to one view decorator, and one test per ❌ cell proves the denial. M3 can generate the authorization test suite directly from this table.

3.4 Django implementation notes (for the developers)

- **Role storage:** use a custom `User` with a `role` field, or a one-to-one `Profile`. Decide before the first migration, since changing it later is painful.
- **Ownership lookup:** `get_object_or_404(CartItem, pk=id, customer=request.user)` . Another user's item then returns 404, so its existence is not leaked.
- **Order placement:** wrap in `with transaction.atomic():` . Read the books inside it, copy the current price into `OrderItem.unit_price` , then clear the cart.
- **Settings checklist:** `AUTH_PASSWORD_VALIDATORS` , `SESSION_COOKIE_SECURE` , `CSRF_COOKIE_SECURE` , `SECURE_SSL_REDIRECT` , `SECURE_HSTS_SECONDS` , `ALLOWED_HOSTS` , `SECRET_KEY` from env. Run `python manage.py check --deploy` before the demo.
- **Throttling:** `django-axes` is the low-effort option for SEC-16.

3.5 Threat review (extends SAD §3.9)

The SAD's seven rows are good. I propose adding these:

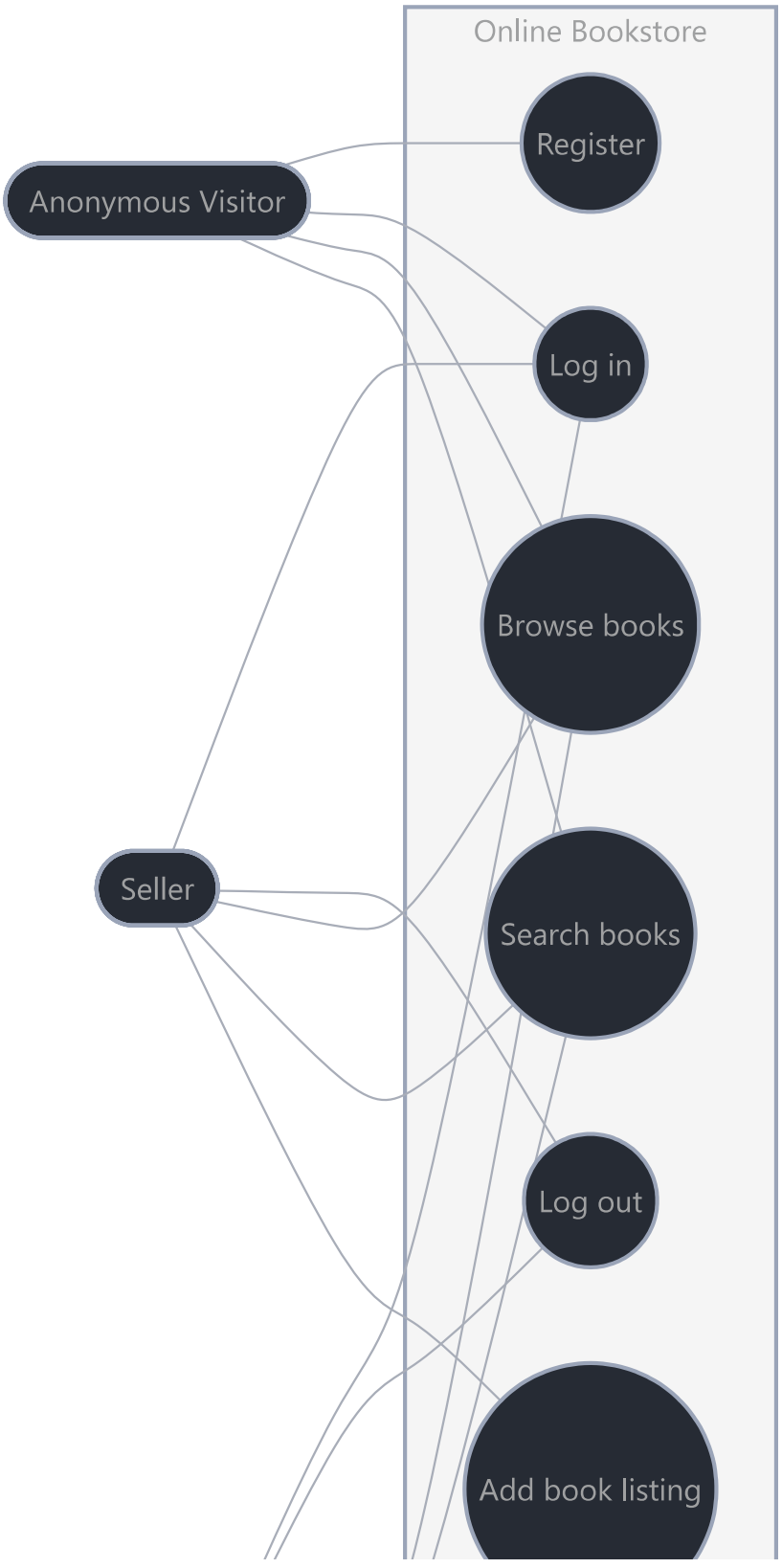
Threat	Example	Control
Brute force / credential stuffing	Script tries many passwords	SEC-16 throttling, SEC-02 password rules
Mass assignment / role tampering	Hidden `role=admin` in registration POST	SEC-07 restricted choices, never expose `is_staff`
Open redirect	`/login/?next=//evil.com`	SEC-20
Session fixation / theft	Reuse of an old session ID	SEC-04, HttpOnly + Secure cookies
IDOR (insecure direct object reference)	`/cart/remove/57/` for someone else's item	SEC-08
Repudiation	"I never placed that order"	SEC-18 logging with user ID and timestamp
Race condition	Two simultaneous order clicks	SEC-11 transaction, PRG redirect

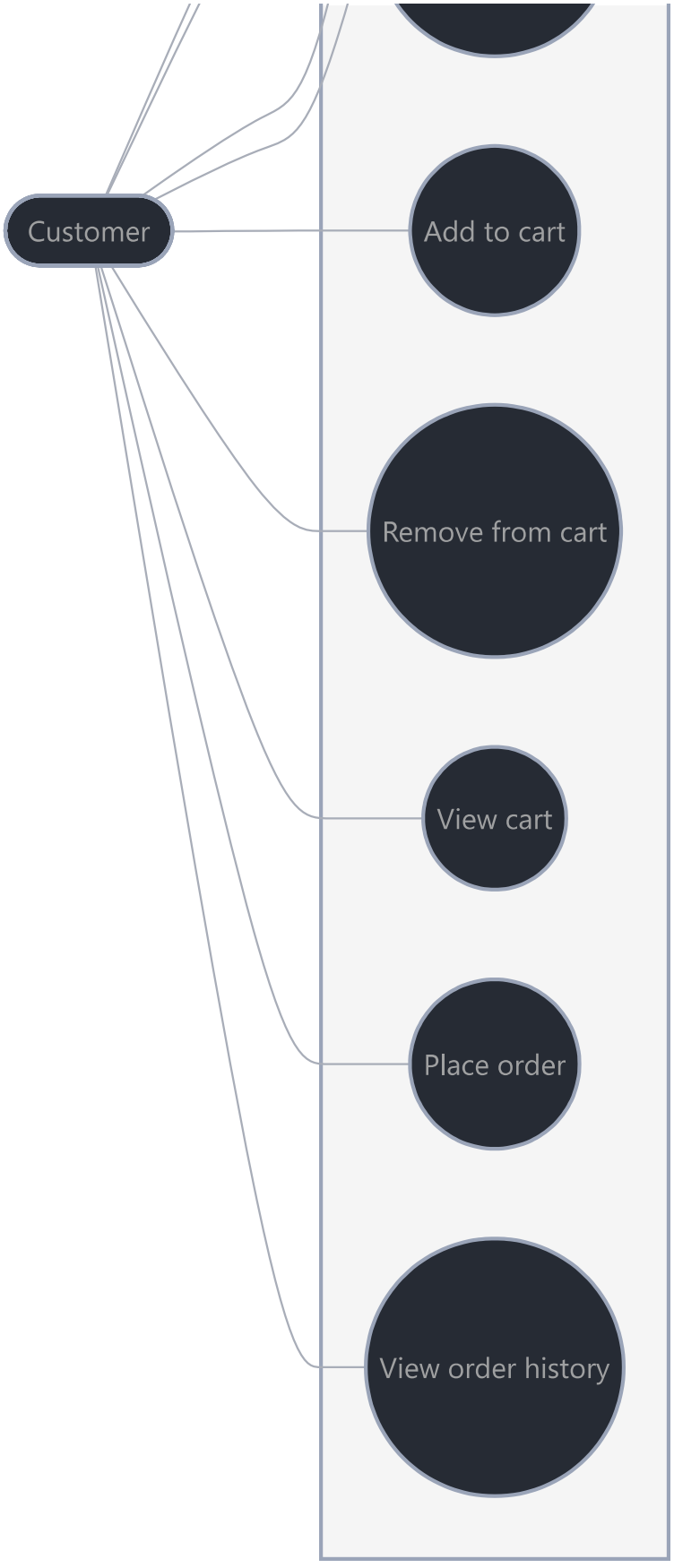
after POST

---

4. Use-Case Diagram

Mermaid has no native UML use-case notation, so this flowchart is a draft for the team. Redraw it in draw.io or PlantUML for the final submission.





UML extras for the final version:

- `Place order` «include» `Validate cart` and `Calculate total`.
- `Add to cart`, `Place order` and `View order history` «include» `Authenticate`.

- Customer and Seller both specialize a generalized actor **Registered User**, if the team agrees that sellers cannot buy.

---

5. Integration Plan

"Integration" here means the four feature modules (`accounts`, `books`, `cart`, `orders`) work together without breaking each other's rules.

5.1 Module contracts

Boundary	Contract
`accounts` → all	Exposes `request.user` and `user.role`, plus the `role_required` decorator. Nobody else reads roles by their own method.
`books` → `cart`	`cart` references `Book` by FK only. It validates existence via the ORM and never copies the price into the cart.
`cart` → `orders`	`orders.place_order(user)` reads the user's cart through a single function (e.g. `cart.services.get_cart_items(user)`) and clears it on success.
`orders` → `books`	Reads the current price and copies it into `OrderItem.unit_price` at purchase time.
All → templates	Shared base template (navigation, messages, error pages).

5.2 Integration order (build and test in this sequence)

1. **accounts:** register, login, logout, role stored. Test SEC-01 to SEC-07.
2. **books:** browse, search, seller add. Test role gate on `/books/add/`.
3. **cart:** add/remove/view. Test ownership and validation (SEC-08, SEC-10).
4. **orders:** place order, history. Test transaction and totals (SEC-09, SEC-11).
5. **End-to-end:** Seller lists a book, Customer finds it, adds it, orders it, and sees it in history. Then run every ❌ cell of the RBAC matrix.

5.3 Integration test scenarios

#	Scenario	Expected
IT-1	Seller adds a book, then a Customer searches for it	Book appears with correct price
IT-2	Seller changes a price after the Customer added the book to the cart	The order uses the new price (server-side)
IT-3	Customer places an order, then views history	Order and items appear, and the cart is empty

IT-4	A failure is forced inside order creation	No Order, no OrderItems, and the cart is intact
IT-5	User A tries to remove or view user B's cart item or order	404, no data change
IT-6	Session expires mid-checkout	Redirect to login, no partial order

---

6. Final Consistency Review

Cross-checking SRS ↔ SAD ↔ Test Plan ↔ diagrams. Issues found in the current SAD draft:

#	Issue	Where	Owner to fix
1	**No data model section.** SAD §2.1 and §3.8 mention a "data model" and schema, but the document has none (no ER diagram or entity/field list).	SAD §3	M2
2	Naming mismatch: Figure 2 says "Book **Management**", §3.4 says "Book **Catalogue**". The same applies to "Authentication" vs "Accounts / Authentication".	Fig. 2 vs §3.4	M2
3	Numbered lists continue across sections (steps 7–20, references 18–20) instead of restarting at 1.	§4.2, §5.2	M2
4	Figure 4 shows the "confirmation" arrow going from Models directly to Browser, skipping the View. Figures 3 and 5 route it through the View.	Fig. 4	M2
5	Section 2.1 says Section 4 covers "testing", but there is no testing section. It should point to the Test Plan.	SAD §2.1	M2/M3
6	Security objectives are TBD in the SRS. Section 1 of this document supplies proposals.	SRS	M1
7	No logout route in the API table, although SEC-04 requires logout.	SAD §4.3	M2
8	Whether a Seller can also browse (yes in the SAD) or purchase is not stated explicitly. The RBAC matrix assumes no.	SRS	M1
9	Whether `Book` records which seller created it (a `seller` FK) is not stated, and it affects audit and any future edit/delete rights.	SAD	M2
10	Traceability matrix uses requirement areas, not IDs. Switch to SEC-xx / FR-xx once M1 and M3 finalize IDs.	SAD §3.8	M3/M1
11	Use-case diagram is still pending (the SAD says so). Section 4 here is the draft.	SAD §4.6	M4

Final consistency checklist (run before submission):

- ⬛ Every SRS requirement appears in the SAD traceability matrix **and** has at least one test case ID.
- ⬛ Every route in SAD §4.3 appears in the RBAC matrix, the use-case diagram, and at least one test.
- ⬛ Every use case in the diagram maps to an SRS functional requirement.
- ⬛ Role names are identical everywhere (Customer, Seller; capitalization too).
- ⬛ Component names are identical in diagrams, tables and code module names.
- ⬛ Every "Proposed" or TBD item is either approved and rewritten as fact, or still listed in Open Issues.
- ⬛ The final code matches the docs (routes, models, permission checks).

---

7. Open Decisions (with my recommendation)

Decision	Recommendation	Why
Role at registration	User picks Customer or Seller from a dropdown. The server validates it and it is not changeable by the user afterward. Admin is created only via `createsuperuser`.	Simple and matches the SRS. Prevents self-escalation.
Can a Seller also buy?	No, for this version.	The SRS describes two distinct roles. A single account with both roles adds complexity.
Database	SQLite for development and demo, PostgreSQL if hosted.	The ORM keeps the code portable.
Login throttling	Include `django-axes` if time allows.	Cheap, and it defends against a common attack.
Duplicate add-to-cart	Increase the quantity on the existing line, with a unique `(customer, book)` constraint.	Easier to test, and the database enforces it.

---

8. Deliverables Checklist (M4)

- ⬛ Security objectives → hand to M1 for SRS (§1)
- ⬛ Security requirements with IDs → hand to M1 and M3 (§2)
- ⬛ Security architecture section → hand to M2 for SAD §3.9 (§3)
- ⬛ Final use-case diagram in draw.io or PlantUML (§4)
- ⬛ Integration order and test scenarios → share with M3 (§5)
- ⬛ Consistency issues → send to each owner and re-check after fixes (§6)
- ⬛ Team review of Open Decisions (§7)